UT 01: Concepto de programa, lenguajes e instalación y uso de entornos de desarrollo.

ÍNDICE

1.- Concepto de programa informático....4
1.1 Ejecución y almacenamiento de los programas....4
1.2 Clasificación funcional de los programas....5
1.3 Características deseables en los programas....5
2. Lenguajes de programación: definición y conceptos....6
3. Clasificación de los lenguajes de programación....7
3.1 Clasificación por niveles de abstracción....7
3.2 Clasificación por la forma de ejecución....9
3.3 Clasificación por el paradigma de programación....9
3.4 Clasificación según la tipificación de los datos....12
3.5 Clasificación según propósito....12
4. Código fuente, código objeto y código ejecutable; máquinas virtuales....12

1.- Concepto de programa informático

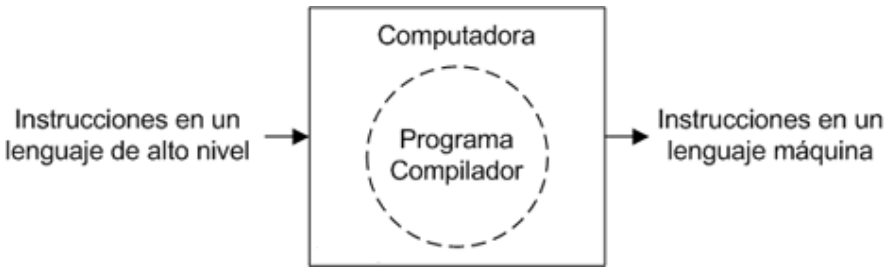
Definición de programa informático: “Un programa informático es un conjunto de instrucciones que se ejecutan de manera secuencial con el objetivo de realizar una o varias tareas en un sistema”.

Un programa informático es creado por un programador en un lenguaje determinado, que será compilado y ejecutado por un sistema. Cuando un programa es llamado para ser ejecutado, el procesador ejecuta el código compilado del programa instrucción por instrucción.

Se podría llegar a decir también que un programa informático es software, pero no seria una definición muy acertada, pues un software comprende un conjunto de programas.

Sin programas, los ordenadores no podrían funcionar. Al conjunto general de programas, se le denomina software, que más genéricamente se refiere al equipamiento lógico o soporte lógico de un ordenador digital.

En informática, se los denomina comúnmente binarios, (propio en sistemas Unix), donde debido a la estructura de este último, los ficheros no necesitan hacer uso de extensiones. Posteriormente, los presentaron como ficheros ejecutables, con extensión (.exe, en los sistemas operativos de la familia Windows) debido a que una vez que han pasado por el proceso de compilación y han sido creados, las instrucciones que se escribieron en un lenguaje de programación que los humanos usan para escribirlos con mayor facilidad, se han traducido al único idioma que la maquina comprende, combinaciones de ceros y unos llamada código máquina.



1.1 Ejecución y almacenamiento de los programas

Típicamente, los programas se almacenan en una memoria no volátil (por ejemplo un disco), para que luego el usuario del ordenador, directa o indirectamente, solicite su ejecución. Al momento de dicha solicitud, el programa es cargado en la memoria de acceso aleatorio o RAM del equipo, bajo el control del software llamado sistema operativo, el cual puede acceder directamente al procesador. El procesador ejecuta (corre) el programa, instrucción por instrucción hasta que termina. A un programa en ejecución se le suele llamar también proceso. Un programa puede terminar su ejecución en forma normal o por causa de un error, dicho error puede ser de software o de hardware.

1.2 Clasificación funcional de los programas

De acuerdo a sus funciones, los programas podemos clasificarlos en software de sistema y software de aplicación.

- El software de sistema es el que se encarga de controlar y gestionar el hardware. Incluye al sistema operativo el cual acopla el hardware con el software de aplicación. El propósito del sistema operativo es proveer un ambiente en el cual el software de aplicación se ejecuta de una manera conveniente y eficiente. Además del sistema operativo, el software de sistema incluye programas utilitarios que ayudan a manejar y configurar la ordenador.

- Si un programa no es software de sistema entonces es software de aplicación. En informática, una aplicación es un tipo de programa informático diseñado como herramienta para permitir a un usuario realizar uno o diversos tipos de trabajo. Esto lo diferencia principalmente de otros tipos de programas como los sistemas operativos (que hacen funcionar al ordenador), las utilidades (que realizan tareas de mantenimiento o de uso general), y los lenguajes de programación (con el cual se crean los programas informáticos).

Suele resultar una solución informática para la automatización de ciertas tareas complicadas como pueden ser la contabilidad, la redacción de documentos, o la gestión de un almacén.

Algunos ejemplos de programas de aplicación son los procesadores de textos, hojas de calculo, y base de datos. Otros ejemplos de programas de aplicación pueden ser: programas de comunicación de datos, multimedia, presentaciones, diseño grafico, cálculo, finanzas, correo electrónico, compresión de archivos, presupuestos de obras, gestión de empresas, etc.

Ciertas aplicaciones desarrolladas «a medida» suelen ofrecer una gran potencia, ya que están diseñadas exclusivamente para resolver un problema específico según las necesidades particulares de un cliente o empresa.

En otras ocasiones algunas compañías agrupan diversos programas de distinta naturaleza para que formen un paquete (llamados suites o suite ofimática) que sean satisfactorios para las necesidades más apremiantes del usuario. Todos y cada uno de ellos sirven para ahorrar tiempo y dinero al usuario, al permitirle hacer cosas útiles con él; algunos con ciertas prestaciones, otros con un determinado diseño; unos son más amigables o fáciles de usar que otros, pero bajo el mismo principio.

1.3 Características deseables en los programas

Corrección. Un programa es correcto si hace lo que debe hacer tal y como se estableció en las fases previas a su desarrollo.
Claridad. Es muy importante que el programa sea lo más claro y legible posible, para facilitar así su desarrollo y posterior mantenimiento.
Eficiencia. Se trata de que el programa, además de realizar aquello para lo que fue creado, lo haga gestionando de la mejor forma posible los recursos que utiliza.
Portabilidad. Un programa es portable cuando tiene la capacidad de poder ejecutarse en una plataforma, ya sea hardware o software, diferente a aquella en la que se elaboró.

2. Lenguajes de programación: definición y conceptos.

En un principio, todos los programas eran creados por el único código que el ordenador era capaz de entender: el código máquina, un conjunto de 0s y 1s de grandes proporciones. Este método de programación, aunque absolutamente efectivo y sin restricciones, convertía la tarea de programación en una labor sumamente tediosa, hasta que fueron apareciendo los diversos lenguajes de programación existentes.

No obstante, nunca hay que olvidar que, usemos el lenguaje que usemos, el compilador hará que de nuestro código solo lleguen 1s y 0s a la máquina.

Se puede definir un lenguaje de programación: como una notación para escribir programas, a través de los cuales es posible establecer comunicación con el hardware y, así, dar las órdenes adecuadas para la realización de un determinado proceso. Viene definido por una gramática o conjunto de reglas que se aplican a un alfabeto constituido por un conjunto de símbolos.

Para estudiar los lenguajes de programación es necesario distinguir tres aspectos:

- Nivel léxico: son los símbolos que se pueden usar en el lenguaje. Por ejemplo: las palabras reservadas del lenguaje (for, class, print, import, etc.), operadores (=, +, -, *, /, etc.), nombres de variables y constantes (contador, dni_alumno, i, x, PI, etc.).
- Nivel sintáctico: las construcciones válidas de esos símbolos. Por ejemplo, el operador de la suma (+) tiene que ir necesariamente acompañado de dos operandos.
- Nivel semántico: el significado de dichas construcciones. Por ejemplo, la instrucción print “hola” genera el código necesario para escribir por pantalla el texto “hola”.

Los elementos básicos de un lenguaje de programación son los siguientes:

- Identificadores: nombres simbólicos que se darán a ciertos elementos de programación (por ejemplo: nombres de variables, tipos, módulos...)
- Constantes: elementos de datos que no cambiarán su valor a lo largo del programa.
- Operadores: símbolos que representan operaciones entre variables, constantes y expresiones.
- Instrucciones: símbolos especiales que representan estructuras de procesamiento y de definición de elementos de programación.
- Comentarios: texto que se usará para documentar los programas.

3. Clasificación de los lenguajes de programación

La cantidad de lenguajes de programación es sencillamente abrumadora, cada uno con unas características y objetivos determinados, por tanto se hace necesario establecer unos criterios para clasificarlos y poder conocerlos mejor. Estos criterios se corresponden con las características principales de los lenguajes.

Se pueden clasificar mediante una gran variedad de formas, se podrían establecer hasta once criterios validos diferentes con los que catalogar un lenguaje de programación. Algunos de dichos criterios pudieran ser redundantes, puesto que se encuentran incluidos explícitamente dentro de otros, como el determinismo o el propósito.

Nosotros vamos a clasificar los lenguajes de programación siguiendo 5 criterios globales y reconocidos: según el nivel de abstracción, por la forma de ejecución, por el paradigma de programación que siguen, por la tipificación de los datos y por último por su propósito.

3.1 Clasificación por niveles de abstracción

Llamamos nivel de abstracción al modo en que los lenguajes se alejan del código máquina y se acercan cada vez más a un lenguaje similar a los que utilizamos diariamente para comunicarnos. Cuanto más alejado esté del código máquina, de mayor nivel será el lenguaje. Dicho de otro modo, podría verse el nivel de abstracción como la cantidad de “capas” de ocultación de código máquina que hay entre el código que escribimos y el código que la máquina ejecutará en último término.

LENGUAJES DE BAJO NIVEL:

(1GL) Primera generación: solo hay un lenguaje de primera generación: el código máquina. Cadenas interminables de secuencias de 1s y 0s que conforman operaciones que la máquina puede entender sin interpretación alguna. Originalmente introducido mediante interruptores. Su uso actual está relegado principalmente a los compiladores, que se encargan de traducir lenguajes de alto nivel a lenguaje máquina. Ejemplo: la secuencia binaria EB11 en la arquitectura x86 produce un salto a la posición relativa 0x11.

LENGUAJES DE MEDIO NIVEL:

(2GL) Segunda generación: los lenguajes de segunda generación tienen definidas unas instrucciones para realizar operaciones sencillas con datos simples o posiciones de memoria. El lenguaje clave de la segunda generación es sin duda el lenguaje ensamblador que permite programar utilizando las instrucciones del microprocesador. Cada instrucción en ensamblador corresponde con una o varias instrucciones en código máquina. Se suele emplear en rutinas que requieren una especial optimización, o para interactuar directamente con el hardware.

Algunos ejemplos de su uso:

Los núcleos (kernel) de los sistemas operativos suelen tener partes escritas en lenguaje ensamblador para aprovechar características de la CPU no accesibles desde lenguajes de más alto nivel.

También se utiliza en la programación de microcontroladores o sistemas empotrados donde se persigue un mayor control sobre la CPU o hay severas limitaciones de memoria.

Muchos juegos de las primeras videoconsolas estaban escritos en ensamblador para sacarle todo el partido al hardware. Ejemplo: la instrucción "MOV EAX, 5" en ensamblador del 80386 asigna el valor 5 al registro EAX.

LENGUAJES DE ALTO NIVEL:

(3GL) Tercera generación: la gran mayoría de los lenguajes de programación que se utilizan hoy en día pertenecen a este nivel de abstracción, en su mayoría, los lenguajes del paradigma imperativo y el de programación orientada a objetos, son lenguajes de propósito general que permiten un alto nivel de abstracción y una forma de programar mucho más entendible e intuitiva, que buscan la aproximación al lenguaje humano, donde algunas instrucciones parecen ser una traducción directa de nuestro propio lenguaje. Por ejemplo, nos podríamos encontrar una línea de código como ésta: IF contador = 10 THEN STOP. No parece que esta sentencia esté muy alejada de cómo expresaríamos en nuestro propio lenguaje Si el contador es 10, entonces para. Cada sentencia en un lenguaje 3GL suele estar asociada a varias instrucciones en código máquina.

Ejemplos: ALGOL, COBOL, Fortran (1950), C, C++, Perl, PHP, Python, Java.

(4GL) Cuarta generación: son lenguajes creados con un propósito específico, que permiten reducir la cantidad de líneas de código que tendríamos que hacer con otros lenguajes de tercera generación mediante procedimientos específicos. Por ejemplo, si tuviésemos que resolver una ecuación en un lenguaje de tercera generación, tendríamos que crear diversos y complejos métodos para poder resolverla, mientras que un lenguaje de cuarta generación dedicado a este tipo de problemas ya tiene esas rutinas incluidas en el propio lenguaje, con lo que solo tendríamos que invocar la instrucción que realiza la operación que necesitamos.

Ejemplos: SQL, PL/SQL, Coldfusion, MATLAB.

(5GL) Quinta generación: también llamados lenguajes naturales, pretenden abstraer más aún el lenguaje utilizando un lenguaje natural con una base de conocimientos que produce un sistema basado en el conocimiento. Pueden establecer el problema que hay que resolver y las premisas y condiciones que hay que reunir para que la maquina lo resuelva. Este tipo de lenguajes los podemos encontrar frecuentemente en inteligencia artificial y lógica.

3.2 Clasificación por la forma de ejecución

Dependiendo de cómo un programa se ejecute dentro de un sistema, podríamos definir tres categorías de lenguajes:

- Lenguajes compilados: un programa traductor (compilador) convierte el código fuente en código objeto y otro programa (enlazador) unirá el código objeto del programa con el código objeto de las librerías necesarias para producir el programa ejecutable.

- Lenguajes interpretados: ejecutan las instrucciones directamente, sin que se genere código objeto, para ello es necesario un programa intérprete en el sistema operativo o en la propia máquina donde cada instrucción es interpretada y ejecutada de manera independiente y secuencial. La principal diferencia con el anterior es que se traducen a tiempo real solo las instrucciones que se utilicen en cada ejecución, en vez de interpretar todo el código, se vaya a utilizar o no.

- Lenguajes virtuales: los lenguajes virtuales tienen un funcionamiento muy similar al de los lenguajes compilados, pero, a diferencia de éstos, no es código objeto lo que genera el compilador, sino un código intermedio o bytecode que puede ser interpretado por cualquier arquitectura que tenga la máquina virtual instalada, la cual se encargará de interpretar el código bytecode generado para ejecutarlo dicha máquina. Aunque de ejecución lenta (como los interpretados), tienen la ventaja de poder ser multisistema o portables y así un mismo código bytecode sería válido para cualquier ordenador independientemente de su sistema operativo, si tiene la máquina virtual instalada.

3.3 Clasificación por el paradigma de programación

Un paradigma de programación es un enfoque particular para la construcción de software, un estilo de programación que facilita la tarea de programación o añade mayor funcionalidad al programa dependiendo del problema que haya que abordar. Todos los paradigmas de programación pertenecen a lenguajes de alto nivel, y es común que un lenguaje pueda usar más de un paradigma de programación.

Lenguajes imperativos: Son lenguajes en los que el programador, para resolver un problema, ha de diseñar un algoritmo y concretarlo en un programa que le diga a la máquina, paso a paso, lo que debe hacer para resolver dicho problema. Se fundamentan en la asignación de variables para almacenar valores y en la realización de operaciones con los datos almacenados.

Los lenguajes imperativos se pueden clasificar, a su vez en:

- Lenguajes Estructurados: son lenguajes que se basan en tres estructuras básicas:
    - Secuencia (una serie de instrucciones que se ejecutan ordenadamente).
    - Condición o Selección (permite la ejecución de unas sentencias u otras en función de una decisión, por ejemplo: si x = 1, imprime "x vale 1", si no, imprime "x no vale 1").
    - Repetición (estructuras "repite mientras ...", "repite hasta que...", etc.).
    - Los primeros lenguajes estructurados fueron ALGOL (muy importante por sus contribuciones al resto de lenguajes), Pascal (muy utilizado en la enseñanza sistemática de la programación), ADA y C.

- Lenguajes No estructurados: carecen de alguna de las estructuras anteriores. Por ejemplo: Fortran (para aplicaciones científicas), R P G (aplicaciones de gestión), o BASIC (propósito general).

Lenguajes declarativos: En estos lenguajes, el programador sólo tiene que declarar o especificar el problema ("qué hay que resolver"), siendo el propio traductor del lenguaje el que se encarga de construir el algoritmo que dará solución al problema ("cómo" resolverlo).

Por ejemplo, en una consulta a una base de datos en SQL:
"select * from articulos where precio > 5", le decimos al intérprete de SQL lo que debe buscar en la tabla (artículos cuyo precio sea mayor que 5), no cómo debe hacerlo.

Otros ejemplos de lenguajes declarativos son LISP o Prolog, éste último está basado en el cálculo de predicados y se emplea para obtener una solución a un problema en base a una serie de reglas lógicas.

Lenguajes procedimentales: con ellos el programa se divide en partes más pequeñas, llamadas funciones y procedimientos, que pueden comunicarse entre sí. Permite reutilizar código ya programado y solventa el problema de la programación spaghetti.

Lenguajes orientados a objetos: crean una estructura de clases y objetos que emula un modelo del mundo real, donde los objetos realizan acciones e interactúan con otros objetos. Los objetos son entidades que tienen un determinado comportamiento (métodos) y una serie de atributos.

Por ejemplo, un objeto Lavadora puede tener los métodos Prelavar, Lavar, Enjuagar y Centrifugar, que corresponden a acciones que puede llevar a cabo el objeto. También almacenará una serie de atributos como tiempo_restante, peso_de_la_carga, y estado_actual.

Ejemplos de lenguajes orientados a objetos: Smalltalk, Java, Python o Ruby.

Algunos lenguajes imperativos se han ampliado para ofrecer características de los orientados a objetos: C++ (es un superconjunto de la sintaxis de C), Objective-C, PHP o Perl.

Respecto a los imperativos, los lenguajes orientados a objetos destacan por su capacidad para reutilizar programas en la construcción de otros nuevos.

Las características más importantes de los lenguajes orientados a objetos son:

- Abstracción: Es la capacidad de un objeto de cumplir sus funciones independientemente del contexto en el que se lo utilice.

  Por ejemplo el objeto Lavadora siempre expondrá sus mismas propiedades y dará los mismos resultados a través de sus eventos, sin importar el ámbito en el cual se lo haya creado.

- Encapsulamiento: Esta característica es la que denota la capacidad del objeto de responder a peticiones a través de sus métodos sin la necesidad de exponer los medios utilizados para llegar a brindar estos resultados.

  Por ejemplo, el método Lavar del objeto Lavadora se pondrá a lavar sin que nosotros tengamos por qué conocer su estado interno, el peso de la carga o cualquier otra propiedad encapsulada en el objeto.

- Herencia: Es la característica por la cual unos objetos pueden adquirir (heredar) los métodos y atributos de sus antecesores, modificándolos y/o añadiendo.

  Por ejemplo, el objeto LavadoraSecadora puede heredar los métodos propios del objeto Lavadora y, a su vez, implementar nuevos métodos relativos a las funciones de secado.

- Polimorfismo: El término de polimorfismo está ligado con la herencia. Define la capacidad de que varias clases derivadas de una antecesora utilicen un mismo método de forma diferente.

  Por ejemplo, un teléfono móvil, uno fijo y otro inalámbrico pueden tener un mismo método Marcar, pero la implementación interna será distinta según el tipo de teléfono.

Lenguajes lógicos o de I.A.: definen un conjunto de reglas lógicas para ser interpretadas mediante inferencias lógicas. Permite responder preguntas planteadas al sistema para resolver problemas.

3.4 Clasificación según la tipificación de los datos

Internamente, el ordenador únicamente es capaz de trabajar con números binarios que se pueden agrupar para formar bytes, cadenas de texto, números de coma flotante o estructuras más complejas. A cada una de las distintas estructuras en las que se pueden agrupar los bits se denomina **tipo de datos**, y cada tipo de datos está asociado con una serie de operaciones. Por ejemplo, dos tipos de datos muy comunes en casi todos los lenguajes de programación son el tipo entero y el tipo cadena (de texto). Un entero se puede sumar, multiplicar, dividir, etc. Por el contrario una cadena se puede concatenar con otra, trocear, invertir, etc.

Los lenguajes utilizan variables y constantes para almacenar los datos que varían a lo largo de la ejecución de un programa. En función del tratamiento de estas variables y constantes y sus tipos tenemos dos clasificaciones de lenguajes:

- Fuertemente tipificados**:** en el momento de escribir el programa, cada variable tendrá asignado un tipo, y generalmente las operaciones entre tipos distintos están prohibidas, siendo responsabilidad del programador hacer las conversiones pertinentes. Por ejemplo, en C, la operación "5" (cadena de texto) + 6 (número entero) está prohibida y sería necesario convertir "5" a entero previamente. Otros ejemplos de lenguajes fuertemente tipificados son: Pascal, C++ o Java.

- Débilmente tipificados**:** el programador no tiene que definir el tipo de las variables, y las conversiones entre éstos suelen hacerse de forma automática por el compilador o intérprete. Por ejemplo, en Perl, la operación anterior ("5" + 6) estaría permitida: el compilador convierte automáticamente la cadena "5" en el entero 5. Otro ejemplo de lenguaje débilmente tipificado: PHP.

3.5 Clasificación según propósito

- Lenguajes de propósito general**:** Los lenguajes de propósito general son los que tienen un amplio dominio de aplicación. Permiten, por tanto, escribir programas para dar solución a problemas de lo más variopinto. Ejemplos de lenguajes de propósito general: C, C++, Java, Perl, PHP, Ruby, Python, Pascal, Cobol, Ensamblador.
- Lenguajes de propósito específico**:** Están especializados en funciones concretas. A destacar:
    - Lenguajes gráficos. Por ejemplo, graphviz.
    - Lenguajes de procesamiento de texto: se utilizan para componer, estructurar y formatear texto e imágenes. Por ejemplo: TeX y LaTeX
    - Lenguajes de consulta de bases de datos. Ejemplo: SQL.
    - Lenguajes para programación web: son lenguajes que, si bien se pueden considerar de propósito general, fueron diseñados

explícitamente para la construcción de aplicaciones web. Por ejemplo: PHP y Coldfusion.

4. Código fuente, código objeto y código ejecutable; máquinas virtuales

Durante la fase de programación se generan distintos tipos de código:

Código fuente: es el escrito directamente por los programadores en editores de texto. Se compone de uno o más ficheros de texto, cada uno de los cuales contiene un conjunto de instrucciones codificadas en algún lenguaje de alto nivel.

Código objeto: es el código binario resultante de compilar el código fuente. Por tanto, el responsable de generar código objeto es el compilador. El código objeto es inteligible para el ser humano (generalmente está en formato binario), sin embargo, tampoco es directamente ejecutable por el ordenador. El compilador genera un fichero de código objeto por cada fichero de código fuente. Aquellos ficheros de código objeto que comparten una finalidad común se suelen agrupar en librerías.

Código ejecutable: es el resultante de enlazar uno o más fragmentos de código objeto con las librerías necesarias. El resultado es un archivo que el sistema operativo es capaz de cargar y ejecutar en memoria. El responsable de construir el ejecutable es el enlazador o linker. Los ficheros con código ejecutable también tienen formato binario, y un ejemplo de ellos en arquitecturas Windows son los archivos con extensiones exe o dll.

Máquinas virtuales: es un entorno de ejecución que actúa como intermediario entre el sistema operativo y el programa. Ejecuta el código previamente compilado (bytecode). Permite programar de manera independiente del sistema, tan sólo es preciso que exista la máquina virtual para este.

El ejemplo más característico es el lenguaje Java. Cualquier sistema que tenga instalado el JRE (Java Runtime Environment) puede ejecutar el código previamente compilado (bytecode).

"A programming language is for thinking about programs, not for expressing programs you've already thought of. It should be a pencil, not a pen"

- Paul Graham